A team of 14 Roblox Studio skills for Claude

What: fourteen agent skills that let Claude or Codex build in Roblox Studio on a Mac like experienced Roblox developers and technical artists, from code architecture and networking to UI, avatar items and monetization. **How:** distilled from 214 top-rated videos and 85 docs, every procedure run live in Roblox Studio 0.743 through its built-in MCP server, then tested blind against an agent without the skills.

214	85	351	14	Studio 0.743	97%
expert videos	docs and articles	expert notes and digests	skills in the team	every procedure run live	blind grade with skills (60% without)

The team

Technical Direction · scenario-roblox-expert
Technical director, lead Roblox engineer: picks the channel (Studio MCP in Edit, playtests, offline Luau checks, Rojo files, headless RunScript, Open Cloud), caps the open place windows and machine load, and routes each brief to a teammate

Code Architecture Gameplay programmer, game architect	**Luau Engineering** Gameplay programmer, Luau specialist	**Networking and Security** Network programmer, anti-exploit engineer	**Player Data** Backend engineer, live-ops programmer	**Tooling and build pipeline** Build engineer, tools programmer
Environment and lighting Builder, environment artist, lighter	**VFX and audio** VFX artist, sound designer	**UI and input** UI designer, UI programmer	**Animation and Characters** Technical animator, character TD	**Gameplay Systems** Gameplay programmer, combat and AI scripter
Asset pipeline and UGC Technical artist, UGC creator	**Performance and Release** Performance engineer, release manager	**Monetization and Live Ops** Monetization designer, live-ops producer		

WHAT EACH SKILL DOES

`scenario-roblox-expert`	Drives Roblox Studio without a mouse through its built-in MCP server, passes each step only on parsed results, frame budgets and captures, and routes each domain to its specialist.	`scenario-roblox-architecture`	Lays out Roblox game code so each script runs once on the right side, boots in two phases, and every connection is cleaned up, proven in live playtests.
`scenario-roblox-luau`	Writes and reviews Luau that stays correct under Deferred events, type-checks on both solvers, frees what it creates, and keeps an optimization only when measured.	`scenario-roblox-networking`	Treats every client call as attacker input: the server owns state, checks each remote in a fixed order, and proves it by firing hostile arguments from a playtest client.
`scenario-roblox-data`	Builds a player data layer that survives crashes, failed writes and server hops: session-locked profiles, safe migrations, one grant per purchase, proven against a simulated multi-server backend.	`scenario-roblox-tooling`	Sets up a Roblox team repository that rebuilds every place from files, gates every pull request with types, lint and tests, and publishes only through CI.
`scenario-roblox-environment`	Builds and lights Roblox maps from code: graybox, terrain, materials, foliage and moods, each checked by probes, voxel reads, image metrics and captures from fixed cameras.	`scenario-roblox-vfx-audio`	Builds layered Roblox effects that the server decides and each client draws, and wires Audio API graphs proven connected and measured, leaving only the listening to a person.
`scenario-roblox-ui`	Builds one Roblox UI tree with StyleSheets and Input Action System contexts, then proves it on nine device presets with layout numbers, real clicks and keys.	`scenario-roblox-animation`	Builds, layers and plays Roblox character, NPC and creature animation in code, then proves each clip, IK chain and face by measured joints on server and client.
`scenario-roblox-gameplay`	Builds NPC AI, melee and projectile combat and constraint cars for Roblox, with the server deciding every hit, rules tested in Lune and server cost measured in a playtest.	`scenario-roblox-assets`	Turns AI-generated or Blender meshes into Roblox assets at true stud size, facing -Z, under budget and watertight, then previews, audits and validates them locally without uploading.
`scenario-roblox-performance`	Finds what makes a Roblox game slow, leaky or crash-prone, proves each fix with before and after numbers against low-end phone budgets, and prepares safe releases.	`scenario-roblox-monetization`	Builds Roblox shops, receipts, passes, odds disclosures, analytics and live config so every purchase grants exactly once after a save and every live change is decided by data.

How the skills were built

Same method as the Blender, Unity and Godot families: about 200 recent, well-rated videos plus the official docs. The value is the **expert delta**: judgment calls, numbers and traps, each checked in the real Studio on a Mac.

Research	Transcripts	Frames	Notes	Version table	Toolkit	Skills	Blind test	Refactor
13 topic clusters, 299 sources ranked by views, likes and authority	214 videos with transcripts	288 contact sheets from 22 videos where the lesson is on screen	301 video and doc notes, 50 digests with a ranked expert delta	45 cited changes up to Studio 0.743 (October 2026)	a Python client for Studio's built-in MCP server, playtests, captures, two-solver type checks	14 skills with Python and Luau kits, every procedure run live in Studio 0.743	12 briefs answered with and without skills, graded blind against the notes	every point both answers missed fed back into its skill, then the live suites rerun

WHERE THE KNOWLEDGE COMES FROM

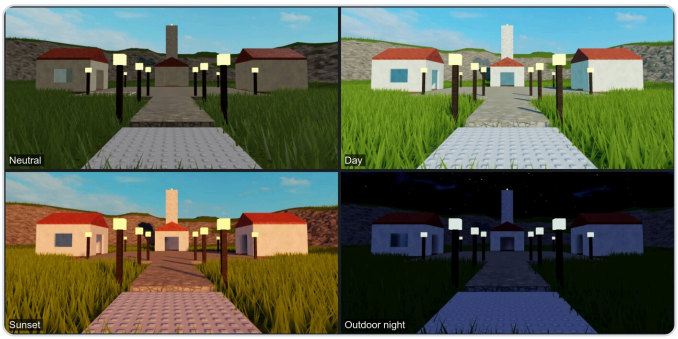
- **Roblox educators:** BrawlDev, Suphi Kaner, Stewiepfing, Crusherfire, DatBoiEle, BONES, Ludius, B Ricey, GnomeCode and others. Channel credentials are as the manifest states them.
- **Roblox staff:** Roblox Learn and Roblox channel videos, RDC talks included (39 and 17 videos).
- **Official material:** 83 Creator Hub documentation pages, 2 articles and 1 talk page.

WHAT "EXPERT DELTA" LOOKS LIKE

> "Under Server Authority, character navigation reports Success and nothing moves." verified live in Studio 0.743
>
> "Attach accessories on the server: a hat added from the client never joins." scenario-roblox-assets, live playtest
>
> "Simulated key presses arrive 0.13 to 0.20 s apart: no input test can spam faster." scenario-roblox-gameplay, live playtest
>
> "A covered Studio window stops drawing: check that a capture changes before judging it." scenario-roblox-expert

HOW AN AGENT DRIVES THE SOFTWARE

- **Studio MCP in Edit:** `execute_luau` with a JSON result protocol, script edits and tree searches (`rbx_mcp`, `rbx_run`).
- **Playtests:** Server and Client code, simulated keys, mouse and walking, captures that are looked at (`rbx_play`).
- **Offline gates:** `rbx_check` with both Luau type solvers, Lune unit tests, AgentKit audits on place files.
- **Files and CI:** Rojo builds, headless RunScript, Open Cloud Luau Execution. A guard caps open place windows at 4 and waits above 85 % GPU or 75 % RAM.



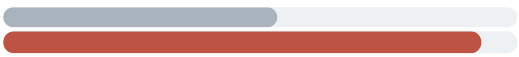
A live capture from the scenario-roblox-environment tests in Studio 0.743.

WHY A TEAM, NOT ONE BIG SKILL

Each skill is one persona's system prompt with explicit handoffs (what it receives and delivers). The lead routes, enforces the protocol and owns the toolkit every skill imports, so several agents can work on one experience and each can be tested alone in the real Studio.

Blind test: the same brief, answered without and with the skills

Twelve briefs were answered by two fresh agents, one without skills and one with them, then graded blind (order shuffled) against the expert notes on five 3-point criteria. Both wrote plans without driving Studio.

Scenario	grey: without skills · colour: with skills	Without	With
G1 Code architecture		**10/15**	**15/15**
G2 Networking and security		**11/15**	**15/15**
G3 Data		**9/15**	**15/15**
G4 Building and environment		**8/15**	**15/15**
G5 Lighting, VFX and audio		**9/15**	**14/15**
G6 UI		**9/15**	**14/15**
G7 Animation and characters		**8/15**	**14/15**
G8 Gameplay systems		**9/15**	**15/15**
G9 UGC and asset pipeline		**7/15**	**14/15**
G10 Tooling and automation		**9/15**	**14/15**
G11 Performance		**10/15**	**15/15**
G12 Monetization and live ops		**9/15**	**14/15**
Total		**108/180 (60%)**	**174/180 (97%)**

Graded answers are written plans; the skills behind them ran live in Studio first. One grader per scenario: a direction, not a statistic. G1 and G8 were regraded once after fixes (13 to 15 each); G10 was graded four times and scored 14 each time; the rest had one pass.

What is verified, what is not yet

Sources and notes	**Done.** 214 videos with transcripts, 288 frame sheets, 85 docs, 50 digests, cited version table.
Skills	**Written, tested and refactored.** 14 skills, each with references, a Python module and Luau kit code.
In Studio 0.743	**Run live.** Every skill's live suite passed in Studio 0.743 on local, unpublished places. Counts are per skill on each sheet.
Not run here	**Needs a published experience or more clients.** Publishing, Marketplace and UGC uploads, purchases, multi-client tests, Team Create, phones and consoles.
Blind grading	**Done.** 60% without skills, 97% with.

HOW TO USE THE SKILLS

- Install from **scenario-labs/skills** once the pull request is merged: `npx skills add scenario-labs/skills` and pick "Expert tools: Roblox Studio", or unzip the skills zip into `~/.claude/skills/`.
- In Studio, open a place and turn on **Enable Studio as MCP server** (Assistant > ... > Manage MCP Servers).
- Start any Roblox task with **scenario-roblox-expert**; it picks the channel and routes to a teammate. One page per skill: `docs/skills/*.pdf`.

NEXT STEPS

- Review and merge the scenario-labs/skills pull request.
- Multi-client tests with StudioTestService:ExecuteMultiplayerTestAsync.
- A published test experience for Open Cloud, purchases, analytics and rewarded video.